

Code Explanation

Data Processing Pipeline Code Explanation

The provided code is a data processing pipeline implemented in Python using Apache Spark. It reads customer and order data from CSV files, cleans and processes the data, and then loads it into Delta tables for further analysis.

Overview of the Code

This code is designed to process customer and order data, perform data cleaning, and load the processed data into Delta tables. It also aggregates customer spend data and loads it into a separate Delta table. The pipeline is implemented using Apache Spark and utilizes Delta Lake for data storage.

Step 1: Get Spark Session

This step creates a Spark session, which is the entry point to any Spark functionality.

```
def get_spark_session():  
    """Get or create a Spark session"""  
    return SparkSession.builder.appName("CustomerOrderDataProcessing").getOrCreate()
```

Step 2: Read Source Data

This step reads customer and order data from CSV files using the Spark session.

```
def read_source_data(spark, customer_path, order_path):  
    """Read source CSV data from volume paths"""  
    customer_df = spark.read.option("header", "true").option("inferSchema", "true").csv(customer_path)  
    order_df = spark.read.option("header", "true").option("inferSchema", "true").csv(order_path)  
    return customer_df, order_df
```

Step 3: Clean Data

This step removes null and duplicate records from the customer and order dataframes.

```
def clean_data(df):  
    """Remove null and duplicate records from dataframe"""  
    df_no_nulls = df.na.drop()  
    df_clean = df_no_nulls.dropDuplicates()  
    return df_clean
```

Step 4: Process Order Data

This step adds a new column "TotalAmount" to the order dataframe by multiplying "PricePerUnit" and "Qty".

```
def process_order_data(order_df):  
    """Add TotalAmount column to order dataframe"""
```

```

        return order_df.withColumn("TotalAmount", col("PricePerUnit") *
col("Qty"))

```

Step 5: Create Delta Tables

This step creates two Delta tables, "ordersummary" and "customeraggregatespend", if they do not already exist.

```

def create_ordersummary_table(spark, catalog, schema):
    """Create ordersummary table if not exists"""
    spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.{schema}.ordersummary (
    CustId STRING,
    Name STRING,
    EmailId STRING,
    Region STRING,
    OrderId STRING,
    ItemName STRING,
    PricePerUnit DOUBLE,
    Qty INT,
    Date DATE,
    TotalAmount DOUBLE,
    IsActive BOOLEAN,
    StartDate TIMESTAMP,
    EndDate TIMESTAMP
) USING DELTA
""")

def create_customeraggregatespend_table(spark, catalog, schema):
    """Create customeraggregatespend table if not exists"""
    spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.{schema}.customeraggregates
pend (
    Name STRING,
    TotalAmount DOUBLE,
    Date DATE
) USING DELTA
""")

```

Step 6: Join and Load to SCD Type 2 Table

This step joins the customer and order data, adds SCD Type 2 columns, and loads the data into the "ordersummary" Delta table.

```

def join_and_load_scd2(spark, customer_df, order_df, catalog, schema
):
    # Join customer and order data
    joined_df = customer_df.join(order_df, "CustId", "inner").select
(
    customer_df["CustId"],
    customer_df["Name"],
    customer_df["EmailId"],
    customer_df["Region"],
    order_df["OrderId"],
    order_df["ItemName"],

```

```

        order_df["PricePerUnit"],
        order_df["Qty"],
        order_df["Date"],
        order_df["TotalAmount"]
    )

    # Add SCD Type 2 columns
    current_time = current_timestamp()
    joined_df_with_scd = joined_df.withColumn("IsActive", lit(True))

    joined_df_with_scd.withColumn("StartDate", current_time)

    joined_df_with_scd.withColumn("EndDate", lit(None))

    # Check if the target table exists and has data
    table_exists = spark._jsparkSession.catalog().tableExists(f"{catalog}.{schema}.ordersummary")

    if table_exists:
        # Update existing records and insert new records
        delta_table = DeltaTable.forName(spark, f"{catalog}.{schema}.ordersummary")
        # ...
    else:
        # First-time load
        joined_df_with_scd.write.format("delta").mode("overwrite")
            .saveAsTable(f"{catalog}.{schema}.ordersummary")

    return joined_df_with_scd

```

Step 7: Aggregate Customer Spend

This step aggregates the "TotalAmount" by "Name" and "Date" from the "ordersummary" Delta table and loads the result into the "customeraggregatespend" Delta table.

```

def aggregate_customer_spend(spark, catalog, schema):
    # Read from ordersummary table
    ordersummary_df = spark.table(f"{catalog}.{schema}.ordersummary")

    # Aggregate data
    aggregated_df = ordersummary_df.filter(col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(spark_sum("TotalAmount").alias("TotalAmount"))

    # Write to customeraggregatespend table
    aggregated_df.write.format("delta").mode("overwrite")
        .saveAsTable(f"{catalog}.{schema}.customeraggregatespend")

    return aggregated_df

```

Step 8: Main Function

This step executes the data processing pipeline by calling the above functions in sequence.

```

def main():
    """Main function to execute the data processing pipeline"""
    spark = get_spark_session()

    # Define paths and catalog/schema
    customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/c
ustomerdata"
    order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata"
    catalog = "gen_ai_poc_databrickscoe"
    schema = "sdlc_wizard"

    # Read source data
    customer_df, order_df = read_source_data(spark, customer_path, o
rder_path)

    # Clean data
    customer_df_clean = clean_data(customer_df)
    order_df_clean = clean_data(order_df)

    # Process order data
    order_df_processed = process_order_data(order_df_clean)

    # Join and load to SCD type 2 table
    join_and_load_scd2(spark, customer_df_clean, order_df_processed,
catalog, schema)

    # Aggregate customer spend
    aggregate_customer_spend(spark, catalog, schema)

    print("Data processing completed successfully")

if __name__ == "__main__":
    main()

```